

get_status

REFERENCE

API consumer

request an on-demand device-health snapshot

Overview

`get_status` retrieves a point-in-time health and status snapshot of the reader. It is a read-only query on the device-management (MGMT) endpoint: the app publishes the command and the device publishes a correlated reply containing power, radio, clock, terminal-link, and battery information under `deviceStatus`.

Because it is read-only, `get_status` is safe to call at any time. It complements the asynchronous `heartBeatEVT` push: `heartBeatEVT` carries only a lightweight inventory-status or battery-alert summary, so use the event stream for continuous monitoring and `get_status` for an immediate, full snapshot.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code>
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.21</code> (API Version Reporting)
Related	<code>get_version</code> , <code>get_current_region</code> , <code>reboot</code> , <code>set_operating_mode</code>

When to Use

Use `get_status` when you need an **immediate, correlated snapshot of reader health** — power source, radio activity/connection, system clock, terminal link, and battery condition — typically in a management workflow over the MGMT endpoint. It confirms device state on demand rather than waiting for the next asynchronous heartbeat.

- **Get:** the current `deviceStatus` snapshot (power, radio, clock, terminal connection, battery).
- **Use it for:** pre-flight safety checks before a state-changing command, spot health checks, and diagnostics. (`deviceStatus.radioActivity` in the response reports the activity status of the

device's radio — see Key Fields.)

- **Prefer a different command when:** you want continuous monitoring rather than a single read — subscribe to `heartBeatEVT` instead for periodic pushes, though it carries only an inventory-status or battery-alert summary, not the full power/radio/clock/terminal/battery snapshot `get_status` returns.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes. To watch a whole fleet rather than one reader, put the MQTT single-level wildcard `+` in the serial slot — `<tenantId>/MGMT/clients/resp/+` — as explained under [Topics and Channels](#).

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `requestId` (request) — your correlation token; make it unique per request. The device echoes it in the response so you can pair reply to request.
- `deviceStatus.radioActivity` (response) — the activity status of the device's radio (`INACTIVE` / `ACTIVE`).
- `deviceStatus.powerSource` (response) — how the reader is currently powered (`WALLCHARGER` / `USB` / `CRADLE` , the contract's enum); current firmware has also been reported to return `NONE` , which the enum does not allow.
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

The request envelope is just `command` and `requestId` — there is no payload wrapper. Use the **Schema** tab for the field reference and the **Example** tab for a ready-to-send, copy-pasteable request.

Request payload

```
1 {
2   "command" : "get_status" ,
3   "requestId" : "abcd123"
4 }
```

Request payload

★ required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	<code>command</code> ★	string	—	<code>get_status</code>	Fixed command name. Used to get the reader status information.
2	<code>requestId</code> ★	string	—	<code>abcd123</code>	A unique identifier for the request, allowing tracking and debugging. Echoed back unchanged in the response.

Response Payload

The response envelope is `command`, `requestId`, `apiVersion`, `response`, and an optional `deviceStatus` snapshot. On the **Schema** tab, expand `deviceStatus` (and its `terminalConnection`, `batteryStatus` sub-objects) to see every field — its type, whether it is required, its enum/units, an example value, and a description; the **Example** tab shows the full JSON. `deviceStatus` is optional at the envelope level; which fields appear depends on device state and model — the **Example** tab shows a representative snapshot.

Response payload

```
1  {
2    "command" : "get_status" ,
3    "requestId" : "abcd123" ,
4    "apiVersion" : "V1.21" ,
5    "deviceStatus" : {
6      "powerSource" : "USB" ,
7      "radioActivity" : "INACTIVE" ,
8      "radioConnection" : "DISCONNECTED" ,
9      "systemTime" : "2026-06-15T09:26:00.443Z" ,
10     "ntp" : {
11       "offset" : 0 ,
12       "reach" : 0
13     } ,
14     "terminalConnection" : {
15       "status" : "CONNECTED" ,
16       "type" : "USB"
17     } ,
18     "batteryStatus" : {
19       "mfgDate" : "22DEC2024" ,
20       "cycleCount" : 8 ,
21       "fullChargeCapacity" : 5739 ,
22       "temperature" : 24 ,
23       "designCapacity" : 6400 ,
24       "batteryType" : "LI-ION" ,
25       "capacity" : 2467 ,
26       "stateOfHealth" : "GOOD" ,
27       "chargePercentage" : 46 ,
28       "chargeStatus" : 1
29     }
30   } ,
31   "response" : {
32     "code" : 0 ,
33     "description" : "Success"
34   }
35 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_status	Echoes the executed command
2	requestId *	string	—	abcd123	Echoes the request's requestId — pairs reply to request.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
3	apiVersion *	enum	V1.0 V1.1 V1.21 V1.22	V1.21	API version the device reports
4	deviceStatus	object	—	—	Status snapshot; optional at envelope level. Which deviceStatus fields appear depends on device state and model — see the Example.
5	powerSource *	enum	WALLCHARGER USB CRADLE	CRADLE	The source of power for the device. The contract's enum WALLCHARGER , USB or CRADLE . Current firmware has also been reported to return NONE (no external power source attached), which is outside enum.
6	radioActivity *	enum	INACTIVE ACTIVE	ACTIVE	The activity status of the device's radio.
7	radioConnection *	enum	CONNECTED DISCONNECTED	CONNECTED	Connection status of the RF radio.
8	hostname	string	—	RFD40-212735201D0053	Device hostname on the network. Optional; may be absent.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
9	system * Time	string	date-time	2022-07-01T03:01:31.464Z	Device system time in ISO
10	temperature	integer	—	32	Device temperature (distinct from the battery temperature below).
11	ntp	object	—	—	Details about the Network Protocol (NTP) synchronization
12	offset	integer	—	120	The offset in milliseconds between the device's clock and the NTP server. Declared in schema, but current firmware does not populate it — the shown is the schema's own example, not a device read. See Response Semantics.
13	reachability	integer	—	321	The reachability score of the NTP server. Declared in the schema, but current firmware does not populate it — the shown is the schema's own example, not a device read. See Response Semantics.
14	terminal connection	object	—	—	Host-terminal link to the slave
15	status	enum	CONNECTED DISCONNECTED	CONNECTED	Terminal connection status

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
16	terminal transport type	enum	BLUETOOTH C10 USB	USB	Terminal transport type.
17	battery status	object	—	—	Battery status information.
18	manufacture date	string	—	22DEC2024	Battery manufacture date.
19	completed battery charge count	integer	—	120	Completed battery charge cycles.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
20	fullChargeCapacity	integer	—	3200	Current full-charge capacity
21	temperature	integer	—	32	Battery temperature (distinct from the device temperature).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
22	design capacity	integer	—	7000	Designed battery capacity.
23	battery Type	string	—	LI-ION	Battery chemistry / type.
24	capacity	integer	—	5000	Remaining battery capacity

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
25	stateOfHealth	enum	GOOD CRITICAL MODERATE	GOOD	Long-term battery capacity health. The same value set the batteryAlert.stateOfHealth carried by alerts and heartBeatEVT — one shared vocabulary, so nothing needs converting. See Response Semantics.
26	chargePercentage	integer	—	23	Percentage of charge remaining
27	chargingStatus	integer	—	1	Charging status: 0 = charger connected; 1 = charger connected, charging; 2 = charger connected, battery 100%.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
28	response *	object	—	—	Result wrapper.
29	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
30	description *	string	—	Success	Human-readable status.

Response Semantics

- **Which `deviceStatus` fields appear depends on device state and model** — absent fields are normal behavior, not an error. Note that `deviceStatus` reports two distinct temperatures: `deviceStatus.temperature` (device) and `deviceStatus.batteryStatus.temperature` (battery) — do not conflate them.
- **The reader has no RTC backup battery.** After a cold start or factory reset, `systemTime` starts at a baseline default; the device syncs its clock from **network time** once it has a network connection. There is currently no API to change the NTP configuration — set the NTP server details (and, if NTP is unreachable, the device time itself) from the **123RFID Desktop** application. Time-sensitive operations (TLS certificate validity, log/event timestamps) may use the default time until the clock has synced.
- **`stateOfHealth` is one grade, on two surfaces.** `deviceStatus.batteryStatus.stateOfHealth` here and the `batteryAlert.stateOfHealth` carried by `alerts` and `heartBeatEVT` use the **same** enum — `GOOD` / `CRITICAL` / `MODERATE` — so shared battery logic needs no conversion. Keep it distinct from the event's `status`, which is a charge band (`LOW` / `CRITICAL` / `CHARGING` / `FULL` / `HIGH`), not a health grade. See Interpret `stateOfHealth`.
- **`ntp.offset` and `ntp.reach` are not reported by the reader.** The schema declares both fields; the values on the **Schema** tab (`120` and `321`) are the schema's own field examples, and the contract's message example shows `0` for each. Current firmware has been reported not to populate either one, so treat their absence as normal and do not build clock-health logic on them. Removing them from the schema is an upstream request to the API owners; until then the fields remain declared.

Response Codes

`get_status` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — status retrieved and returned.
3	Not able to retrieve information — the reader could not gather the requested status at this moment. Retry; if persistent, reboot after stopping any active inventory.
101	Default error — generic catch-all.

Examples

`get_status` is a single request/response exchange. Publish the standard `{command, requestId}` request shown under [Request Payload](#); the device replies on the response topic with the correlated reply — the `requestId` you send (`abcd123`) is echoed back so you can pair reply to request. The full reply, field by field, is shown under [Response Payload](#) above.

Notes and Caveats

- `get_status` is the on-demand poll; `heartBeatEVT` and related alerts push a lighter periodic summary (an inventory-status or battery-alert block), not the full `get_status` snapshot. Prefer the events for continuous monitoring and `get_status` for a complete point-in-time check. Cross-command `apiVersion` reporting can differ (for example `mqttConnEVT` emits a numeric `apiVersion`); that is a firmware note, not a `get_status` schema difference.

Related: [The Configuration Model](#) · [Response Codes](#) · [MQTT API](#)